Part 1: Git Deep Dive

1. Anatomy of Git Architecture

Git operates across four primary areas:

- **Working Directory:** Your local filesystem where you edit files live.
- **Staging Area (Index):** A draft buffer where you prepare changes before recording them.
- **Local Repository (.git folder):** The local database storing all committed historical snapshots.
- **Remote Repository (GitHub):** The hosted central repository where the team collaborates.

```
None
[Working Directory] --(git add)--> [Staging Area] --(git commit)--> [Local Repo] --(git push)--> [Remote Repo]
```

2. Daily Command Cheat Sheet

Bash

```
None
git status                    # Check modified, staged, and untracked files
git add path/to/file.ts       # Stage specific file (git add . stages all)
git commit -m "feat: add cart modal" # Create a snapshot of staged changes
git log --oneline --graph     # View clean visual commit history
git diff                      # Show unstaged changes in working directory
git fetch origin              # Download latest metadata from remote without merging
git pull origin main          # Fetch AND merge remote main into active branch
git push origin feature/cart  # Upload local commits to remote repository
```

3. Branching Strategy & Merging

Never commit directly to main. main represents production-ready code.

- **Create and Switch:**
- Bash

```
None
git checkout -b feature/product-card
# Modern alternative: git switch -c feature/product-card
```

-
- **Merging Workflow:**
- Bash

```
None
git checkout main
git pull origin main
git merge feature/product-card
```

-
- **Branch Deletion:**
- Bash

```
None
git branch -d feature/product-card # Delete locally
```

-

4. Resolving Merge Conflicts

Merge conflicts occur when two branches modify the exact same lines of code in different ways, and Git cannot automatically determine which version is correct.

- **Conflict Markers in Editor:**
- TypeScript

```
None
<<<<<<< HEAD (Current Change - main)
const cartTotal = items.reduce((acc, item) => acc + item.price, 0);
=======
const cartTotal = calculateSubtotal(items) + tax;
>>>>>>> feature/cart (Incoming Change)
```

-
- **Resolution Steps:**
    1. Open the file, inspect both code blocks, and delete the unwanted code along with the `<<<<<<<`, `=======`, and `>>>>>>>` markers.
    2. Stage the resolved file: `git add src/cart.ts`.
    3. Complete the merge commit: `git commit -m "fix: resolve cart total merge conflict"`.

5. revert vs reset & stash

- **git revert <commit-id>:** Creates a **new commit** that undoes the changes of a previous commit. It is safe for shared/public branches because it preserves commit history.
- **git reset --hard <commit-id>:** Rewinds history and **wipes out changes** after that commit. Never use this on shared remote branches!
- **git stash:** Temporarily shelves uncommitted changes so you can switch branches with a clean working copy.
    - `git stash` saves changes.

    - git stash pop restores the most recently stashed changes.

6. Secrets Management & .gitignore

- .gitignore tells Git which files, build artifacts, or secret keys to ignore (e.g., node_modules/, .next/, .env.local).
- **Critical Rule:** Never commit API secret keys, database password URIs, or tokens. Once pushed to GitHub, assume public disclosure even in private repos. Keep all environment secrets in .env.local.

7. Clean Commit Protocol

- **Atomic Commits:** Make small, single-purpose commits (e.g., separate a styling commit from a data fetching commit).
- **Format:** Use imperative conventional commit messages:
    - feat: add product sorting dropdown
    - fix: correct calculation on cart discount
    - docs: update API setup instructions in README

8. Pull Requests (PRs) & Code Review

A Pull Request notifies team members that a feature branch is ready for review. It allows reviewers to comment on specific lines of code, request adjustments, run automated checks, and approve before merging into main.

Part 2: Web Fundamentals

1. Frontend vs Backend (Execution Environments)

- **Frontend (Client-side):** Runs inside the browser (HTML, CSS, JavaScript/TypeScript). Handles UI rendering, user input capture, animations, and local state.
- **Backend (Server-side):** Runs on remote hardware/containers (Node.js, Python, PostgreSQL). Handles business logic, database queries, authentication checks, and secret API key security.

2. What Happens When You Open a URL?

When you enter https://store.com/products:

1. **DNS Lookup:** Browser queries DNS servers to translate the human domain (store.com) into an IP address (e.g., 192.0.2.1).
2. **TCP/TLS Handshake:** Establishes a secure connection between client and server.
3. **HTTP Request:** Browser sends a GET /products HTTP/1.1 request with headers.
4. **Server Processing:** Server validates the request, queries the database, and builds a response.
5. **HTTP Response:** Server replies with status 200 OK and a payload (HTML or JSON).
6. **Critical Rendering Path:** Browser parses HTML to construct DOM tree → applies CSS to build CSSOM → executes JavaScript → paints pixels on screen.

3. HTTP Methods & Status Codes

Common HTTP Methods

- GET: Retrieve data (no request body).
- POST: Submit new data to create a resource.
- PUT: Replace an existing resource entirely.
- PATCH: Partially update an existing resource.
- DELETE: Remove a resource.

HTTP Status Codes

- **2xx Success:** 200 OK, 201 Created
- **4xx Client Errors:** 400 Bad Request, 401 Unauthorized (not logged in), 403 Forbidden (logged in, lacking permission), 404 Not Found
- **5xx Server Errors:** 500 Internal Server Error

4. REST APIs, JSON, and Request Headers

- **REST (Representational State Transfer):** An architectural standard using URIs to identify resources (e.g., GET /api/v1/products/42).
- **JSON (JavaScript Object Notation):** The universal lightweight data interchange format:
- JSON

```
None
{
 "id": 42,
 "name": "Wireless Mouse",
 "price": 29.99,
 "inStock": true
}
```

-
- **Request Headers:** Metadata sent with requests (e.g., Content-Type: application/json, Authorization: Bearer <token>).

5. CORS (Cross-Origin Resource Sharing)

CORS is a browser security mechanism that blocks web pages from making API requests to a different domain (origin) than the one serving the page.

- **Origin Definition:** Protocol + Domain + Port (e.g., http://localhost:3000 vs http://api.myapp.com:5000 are separate origins).
- **Fixing CORS:** The backend server must return specific response headers granting access:
- HTTP

```
None
Access-Control-Allow-Origin: http://localhost:3000
```

```
Access-Control-Allow-Methods: GET, POST, OPTIONS
```

-

6. Environment Variables

- Environment variables separate application code from environment-specific configuration (development, staging, production).
- .env.local remains on your local machine and is ignored by Git.
- Secrets like SUPABASE_SERVICE_ROLE_KEY or DATABASE_URL must **never** be prefix-exposed to client-side bundles.

Part 3: React Core Fundamentals

1. Components, JSX, Props, and State

- **Components:** Independent, reusable pieces of UI. Think of them as custom JavaScript functions that return HTML-like code (JSX).
- **JSX:** JavaScript XML allows you to write HTML tags directly inside JavaScript files.
- **Props (Properties):** Read-only data passed *down* from a parent component to a child component (like function arguments).
- **State:** Data managed *inside* a component that changes over time based on user interactions. When state changes, React automatically re-renders the component to update the UI.

2. Rendering Lists and the key Prop

When mapping over arrays to display lists (like a product catalog), every rendered item **must** have a unique key prop.

TypeScript

```
None
<ul>
  {products.map((product) => (
    <li key={product.id}>{product.name} - ${product.price}</li>
  ))}
</ul>
```

-

  Why key matters: React uses keys to track which items have changed, been added, or been removed. Without stable unique keys, React's reconciliation algorithm gets confused, leading to sluggish re-renders, bugs, or broken form states. Never use array index (index) as a key if items can reorder or filter.

3. Events and Controlled Inputs

- **Events:** Handled via camelCase syntax (e.g., onClick, onChange, onSubmit).
- **Controlled Inputs:** Form elements whose value is entirely driven by React state:
- TypeScript

```
None
const [email, setEmail] = useState("");
return (
  <input
    type="email"
    value={email}
    onChange={(e) => setEmail(e.target.value)}
  />
);
```

-

4. Essential Hooks

- **useState:** Adds reactive state to functional components. Returns the current state value and a setter function.
- **useEffect:** Handles side effects (fetching data, subscribing to events, modifying the DOM directly) after a component renders.
    - **Dependency Array Rules:**
        - useEffect(() => { ... }): Runs after **every** render.
        - useEffect(() => { ... }, []): Runs **once** on initial component mount.
        - useEffect(() => { ... }, [productId]): Runs on mount **and** whenever productId changes.
- **useRef:** References a mutable DOM element directly or stores a variable value that persists across re-renders without triggering a re-render when changed.

5. Lifting State Up

When two sibling components need to share the same data (e.g., a **Product List** that needs to add items to a **Shopping Cart**), you must "lift" that state up to their closest common parent component. The parent holds the state (cartItems) and passes down the state and updater functions (addToCart) via props.

Part 4: React + Separate Backend vs. Next.js

1. Traditional Separate Architecture (Vite + Express + MongoDB)

- **How it works:** Your frontend (Vite React app running on localhost:5173) runs completely separate from your backend API server (Express/Node.js running on localhost:5000).

- **Communication:** They talk to each other strictly over HTTP requests (fetch or axios), requiring strict CORS configuration on the backend server.

2. What Next.js Adds

Next.js is a full-stack framework built on top of React that eliminates the need for a separate backend server for most standard web applications:

- **File-Based Routing:** Every file inside app/ automatically becomes a route (e.g., app/products/page.tsx → domain.com/products).
- **Server Components (RSC) vs. Client Components ('use client'):**
    - By default, all Next.js App Router components are **Server Components**. They run exclusively on the server, can fetch data directly from databases without exposing credentials, and send zero JavaScript to the browser.
    - **Client Components** run in the browser, enabling interactivity, hooks (useState, useEffect), and browser event listeners.
- **Route Handlers:** Build server-side API endpoints (app/api/products/route.ts) natively inside Next.js.

3. CSR vs. SSR vs. Static Generation

- **CSR (Client-Side Rendering):** Browser downloads an empty HTML skeleton, then JavaScript runs to fetch data and render the UI. (Standard Vite React apps).
- **SSR (Server-Side Rendering):** The server generates the full HTML page on every single request with fresh data, then sends it to the browser. Great for dynamic, authenticated dashboards.
- **SSG (Static Site Generation):** Pages are built once at compile/build time. Lightning fast, ideal for blogs or documentation sites.

4. When a Separate Backend Still Makes Sense

Even with Next.js, complex enterprise systems sometimes require a separate backend (e.g., heavy microservices architecture, long-running background worker queues, or APIs consumed by native mobile apps and third-party clients).

5. Environment Variables (NEXT_PUBLIC_)

- Standard variables (SUPABASE_SERVICE_KEY) are **server-only** and completely hidden from the browser.
- Variables prefixed with NEXT_PUBLIC_ (NEXT_PUBLIC_SUPABASE_URL) are intentionally exposed to the client-side browser bundle during build time. Never put private secrets behind a NEXT_PUBLIC_ prefix!